01 — EXECUTIVE SUMMARY

## A video studio that runs on the laptop, not on a subscription

FrameForge Studio is a desktop application that turns a written brief into a finished MP4 using **only the user's own language-model key**. The model writes a multi-scene storyboard as animated HTML/CSS, an offscreen Chromium renderer draws every frame, and FFmpeg encodes the film on the same machine. There is no backend to host and no per-second video API in the path.

It removes a real cost: a short product film today means a subscription editor or a generative-video API billed by the second (about **$0.05–$0.10 per second**), plus a server holding the customer's keys. FrameForge replaces that with one OpenAI-compatible key — DeepSeek, OpenRouter, Groq or a local Ollama.

| $0.00 | 0 | 1 | 7 |
|---|---|---|---|
| VIDEO API SPEND PER RENDER | BACKEND SERVERS TO OPERATE | API KEY, USER-SUPPLIED | PIPELINE STAGES, ON-DEVICE |

02 — TECH STACK & HOW IT IS BUILT

## Electron shell, HyperFrames UI, FFmpeg engine, OS-native vault

| LAYER | IMPLEMENTATION | WHERE |
|---|---|---|
| **Desktop framework** | Electron 33 + electron-vite 2, split into main, preload and renderer. The renderer is sandboxed with `contextIsolation`, no Node integration and a `default-src 'none'` CSP; packaged per-user as an NSIS installer. | `src/main/index.ts`<br>`electron.vite.config.ts` |
| **UI layer** | **HyperFrames UI** — `@hyperframes/studio` 0.9.30 primitives (Tabs, Button, Input, Select, Toggle, Tooltip) on HyperFrames tokens, plus `@hyperframes/player` for preview. React 19, Tailwind 4, Lucide. | `src/renderer/src/**`<br>`package.json` |
| **Rendering engine** | **FFmpeg / ffprobe** driven from the main process: probe, normalise, `libx264` + `yuv420p` + CRF 20 + `+faststart`, concat-demuxer cuts or an `xfade` chain, poster extraction. | `src/main/ffmpeg/**`<br>`src/main/render/cards.ts` |
| **Storyboard & deck** | **OfficeCLI** 0.2.121 as a child process with a sanitised environment, invoked `--json` to build the delivery `.pptx` (and `.docx` path). Absent OfficeCLI skips the stage with a stated reason. | `src/main/office/index.ts` |
| **Credential handling** | **OS credential store** via `@napi-rs/keyring` (Credential Manager / Keychain / libsecret), falling back to Electron `safeStorage` (DPAPI / Keychain), then session memory. Disk holds last-4 and a SHA-256 fingerprint only. | `src/main/vault.ts` |
| **Model integration** | Any OpenAI-compatible `POST /chat/completions`: **DeepSeek** (`deepseek-chat`), OpenRouter, Groq, Ollama presets or a custom base URL, each allowlisted to its own host. Zod contracts re-validated at every IPC boundary. | `src/main/providers/llm.ts`<br>`src/shared/contract.ts` |

03 — KEY FEATURES

## What a producer can do from one window

- **Studio — brief to video**
  - **Brief-driven:** one written brief is the whole direction; the model returns 2–8 scenes.
  - **Format control:** 16:9 / 9:16 / 1:1, 8–30 s total, scene count and per-scene timing derived for you.
  - **Joins & bookends:** crossfade or hard cut, branded opening/closing cards with headline, subhead and accent theming, previewed live.
- **Jobs queue — render tracking**
  - **Serial queue** persisted as JSON; jobs survive restarts with their full history.
  - **Stage-accurate progress** weighted across the seven stages (3 / 7 / 44 / 12 / 10 / 19 / 5), with per-scene detail.
  - **Diagnostics:** the failing stage carries the model's or FFmpeg's own message; cancel at checkpoints.
- **Keys & System — trust surface**
  - **Local key management** with presets, last-4, fingerprint and a zero-token **Test** (`GET /models`).
  - **Toolchain health:** FFmpeg, ffprobe, OfficeCLI, HyperFrames, Electron/Chromium/Node, vault backend, data directory.
  - **Network posture** stated explicitly behind a "no backend" pill.
- **Deliverables — ready to hand over**
  - **Video:** `.mp4`, H.264 / yuv420p / 30 fps / +faststart, plus a poster JPG.
  - **Record:** `storyboard.json` with title, duration and HTML size per scene.
  - **Deck:** optional `.pptx` from OfficeCLI; every artifact listed with size and absolute path.

04 — PIPELINE ARCHITECTURE

## Seven stages, run in order and observable from the Jobs view

| 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|---|---|---|---|---|---|---|
| **Validate** | **Storyboard** | **Generate shots** | **Download** | **Compose cards** | **Render** | **Export** |
| Toolchain + model + brief | LLM → JSON / HTML | Chromium capture | Skipped, nothing to fetch | Opening / closing | FFmpeg normalise + join | MP4 · poster · JSON · deck |

| # | STAGE | WHAT HAPPENS | TIME |
|---|---|---|---|
| 01 | **Validate inputs & toolchain** | Confirms FFmpeg/ffprobe are present, a model is configured and the brief is not empty. Nothing is spent before this passes. | instant |
| 02 | **Storyboard (LLM)** | One `/chat/completions` call returns 2–8 scenes, each a self-contained animated HTML document; durations are rescaled to the target length. OfficeCLI turns the same scene list into the `.pptx/.docx`. | 5–25 s |
| 03 | **Generate shots (local capture)** | Each scene is sanitised — scripts, frames, embeds, inline handlers and remote references removed — then wrapped in the app's pause/seek driver, captured frame by frame offscreen and encoded. A bad scene falls back to a branded card. | 4–10 s / scene |
| 04 | **Download source clips** | Skipped by design: scenes are rendered locally, so there is no vendor clip to fetch. The stage remains in the model for a future provider-backed mode. | 0 s |
| 05 | **Compose title / bookend cards** | Opening (3.2 s) and closing (2.8 s) brand cards rendered from the same template as the preview, captured to PNG sequences and encoded. | 6–12 s |
| 06 | **Render with FFmpeg** | Every segment is normalised to one pixel format, size and frame rate, then joined by concat demuxer (cut) or xfade chain (crossfade); output H.264 / yuv420p / CRF 20. | 4–10 s |
| 07 | **Export deliverables** | ffprobe records duration and size, a poster is extracted at 1.5 s, the scene record is written and the optional deck generated; every artifact is registered on the job. | 1–3 s |

**Verified on this machine:** a 12-second brief produced 3 scenes and a 15.6 s, 1920×1080 H.264 film (2.4 MB) with poster and scene record — video-API spend **$0.00**.

END-TO-END TESTED

05 — SECURITY & COST EFFICIENCY

## Local-first by construction, not by policy

### Local-first execution

- No FrameForge server, no telemetry; the renderer has no network access at all.
- Sandboxed renderer with denied navigation, window-open and permission requests.
- Model-written HTML is untrusted data, captured under `default-src 'none'`.

### BYO-model flexibility

- One key for any OpenAI-compatible endpoint — DeepSeek, OpenRouter, Groq or a fully local Ollama.
- Model and base URL are user settings, not hard-coded; swapping models needs no rebuild.
- The allowlist is derived per client from that base URL, so no second host is implicitly trusted.

### Encrypted keys & cost

- Keys live in the OS credential store; `safeStorage` is the fallback, memory-only the last resort, and the UI names which is in use.
- Disk carries last-4 and a SHA-256 fingerprint only; the renderer can never read a key back.
- Render cost is $0.00 — the only spend is a few thousand LLM tokens per storyboard.

GENERATIVE VIDEO API

**$0.05–0.10** per second

Rates already modelled in the app's own estimate table.

COST OF ONE 15-SECOND FILM

**≈ $0.75–1.50**

Billed by the second, on every render, before any seat licence.

FRAMEFORGE STUDIO

**$0.00**

Video cost is zero; the only spend is a few thousand LLM tokens.

06 — CURRENT CAPABILITIES & NEXT STEPS

## Production-ready for internal use; external distribution needs signing

### Working today

- ✓ v0.1.0 Windows installer, per-user with no admin prompt; desktop and Start Menu shortcuts.
- ✓ Strict TypeScript build and 72 automated tests across 9 files, including real FFmpeg renders.
- ✓ Free pipeline proven on the installed build; 13-page operator manual generated from the same codebase.

### Next enhancements

- → Validate against a live paid model account — current runs used a local mock endpoint.
- → Audio bed and voiceover (output is silent by design today).
- → macOS/Linux builds, code signing and a branded executable; template presets and batch rendering.